

Claude-এ Plugin ব্যবহারের সম্পূর্ণ বাংলা টিউটোরিয়াল

ইনস্টলেশন, ব্যবস্থাপনা এবং নিজের Plugin তৈরির হাতেকলমে নির্দেশিকা

Plugin হলো Claude Code-এর ক্ষমতা বাড়ানোর সবচেয়ে সহজ উপায় — একটি প্যাকেজেই থাকে skill, agent, hook, MCP server আর LSP server। এই বইটিতে ধাপে ধাপে দেখানো হয়েছে কীভাবে marketplace যোগ করবেন, প্রয়োজনীয় plugin ইনস্টল করবেন, দল বেঁধে ব্যবহার করবেন এবং শেষ পর্যন্ত নিজের plugin বানিয়ে অন্যদের সঙ্গে ভাগ করে নেবেন।

CLI

Desktop App

Web / Cloud

সংস্করণ ১.০ · সেপ্টেম্বর ২০২৬

তথ্যসূত্র: code.claude.com/docs

সূচিপত্র

অধ্যায় ১ — Plugin কী এবং কেন দরকার	5
১.১ সংজ্ঞা	5
১.২ Plugin না standalone কনফিগারেশন?	5
১.৩ Plugin ব্যবহারের সুবিধা	6
১.৪ খরচের দিকটিও মনে রাখুন: Context cost	6
অধ্যায় ২ — Plugin-এর উপাদানসমূহ	7
২.১ এক নজরে সব উপাদান	7
২.২ Skill	8
২.৩ Agent (Subagent)	8
২.৪ Hook	9
২.৫ MCP server	9
২.৬ LSP server	10
২.৭ তিনটি environment variable	10
অধ্যায় ৩ — Marketplace কীভাবে কাজ করে	11
৩.১ দুই ধাপের মডেল	11
৩.২ Anthropic-এর তিনটি marketplace	11
৩.৩ Marketplace যোগ করার চারটি উপায়	11
অধ্যায় ৪ — অত্যাবশ্যকীয় Plugin-এর তালিকা	13
৪.১ Code intelligence (LSP) plugin	13
৪.২ External integration — বাইরের service-এর সংযোগ	14
৪.৩ নিরাপত্তা	14
৪.৪ Development workflow	15
৪.৫ Output style	15
৪.৬ কোনগুলো দিয়ে শুরু করবেন	15

অধ্যায় ৫ — ধাপে ধাপে Plugin সংযোজন	16
৫.১ Claude Code CLI (টার্মিনাল)	16
৫.২ Claude Desktop app	18
৫.৩ Web ও Cloud session	18
৫.৪ তিনটি পরিবেশের তুলনা	19
অধ্যায় ৬ — ব্যবস্থাপনা, Scope ও হালনাগাদ	20
৬.১ চারটি scope	20
৬.২ দৈনন্দিন command	20
৬.৩ Installed ট্যাব	21
৬.৪ Marketplace ব্যবস্থাপনা	21
৬.৫ স্বয়ংক্রিয় হালনাগাদ	22
৬.৬ ইনস্টলের সময় ক্যাটালগ হালনাগাদের নিয়ম	22
৬.৭ Reload-এর খরচ	23
অধ্যায় ৭ — নিজের Plugin তৈরি	24
৭.১ প্রথম Plugin — হাতেকলমে	24
৭.২ Skills ডিরেক্টরিতে plugin রাখা	26
৭.৩ পূর্ণাঙ্গ ডিরেক্টরি কাঠামো	26
৭.৪ plugin.json — পূর্ণ schema	27
৭.৫ পরীক্ষা ও ত্রুটি নির্ণয়	28
৭.৬ পুরোনো কনফিগারেশনকে plugin-এ রূপান্তর	28
অধ্যায় ৮ — দলগত Marketplace ও নিরাপত্তা	30
৮.১ দলের জন্য marketplace কনফিগার করা	30
৮.২ প্রতিষ্ঠানের নিয়ন্ত্রণ	30
৮.৩ নিজের marketplace তৈরি ও বিতরণ	31
৮.৪ Community marketplace-এ plugin জমা দেওয়া	31
অধ্যায় ৯ — সমস্যা সমাধান	33

৯.১ /plugin command চেনা যাচ্ছে না	33
৯.২ ইনস্টল ব্যর্থ হচ্ছে	33
৯.৩ Skill দেখা যাচ্ছে না	34
৯.৪ Code intelligence (LSP) সংক্রান্ত সমস্যা	34
৯.৫ Diagnostics নিজে পড়া	34
পরিশিষ্ট ক — Command রেফারেন্স	35
Session-এর ভেতরের command	35
Shell command	35
পরিশিষ্ট খ — পরিভাষা	37

এই বইয়ের সব command ও ফাইলের নাম ইংরেজিতে রাখা হয়েছে, যাতে আপনি সেগুলো ছবছ কপি করে টার্মিনালে ব্যবহার করতে পারেন।
ব্যখ্যা ও নির্দেশনা বাংলায়।

Plugin কী এবং কেন দরকার

Claude Code দিয়ে কাজ করতে করতে প্রায় সবাই একটা জায়গায় পৌঁছান — কিছু কাজ বারবার একইভাবে করতে হয়। প্রতিবার একই নির্দেশনা টাইপ করা, একই review checklist মনে রাখা, একই external service-এর সঙ্গে যুক্ত হওয়া। Plugin ঠিক এই সমস্যাটির সমাধান।

১.১ সংজ্ঞা

Plugin হলো একটি স্বয়ংসম্পূর্ণ (self-contained) প্যাকেজ, যা Claude Code-এর সঙ্গে নতুন সক্ষমতা যুক্ত করে। একটি plugin-এর ভেতরে একসঙ্গে থাকতে পারে:

- **Skill** — নির্দিষ্ট কাজের জন্য লেখা নির্দেশনা, যা Claude প্রয়োজন বুঝে নিজেই ব্যবহার করে
- **Agent** — আলাদা ব্যক্তিত্ব ও সীমাবদ্ধতাসহ বিশেষায়িত subagent
- **Hook** — নির্দিষ্ট ঘটনার সময় স্বয়ংক্রিয়ভাবে চলা script
- **MCP server** — GitHub, Slack, Figma-র মতো বাইরের service-এর সঙ্গে সংযোগ
- **LSP server** — কোড বোঝার জন্য language server, যা type error সঙ্গে সঙ্গে ধরিয়ে দেয়

গুরুত্বপূর্ণ কথা হলো, এই সবকিছু একটি মাত্র ফোল্ডারে থাকে এবং একটি মাত্র command দিয়ে ইনস্টল করা যায়। ফলে আপনি যা তৈরি করলেন, তা সহজেই সহকর্মীদের সঙ্গে ভাগ করে নেওয়া যায় — তাদের আর হাতে ধরে কনফিগার করতে হয় না।

১.২ Plugin না standalone কনফিগারেশন?

Claude Code-এ custom skill, agent বা hook যোগ করার দুটি পথ আছে। প্রকল্পের `.claude/` ফোল্ডারে সরাসরি ফাইল রাখা যায়, আবার সেগুলোকে plugin আকারেও গোছানো যায়। কোনটি কখন বেছে নেবেন:

পদ্ধতি	Skill-এর নাম	কখন উপযুক্ত
Standalone <code>.claude/</code> ডিরেক্টরি	<code>/hello</code>	ব্যক্তিগত workflow, একটিমাত্র প্রকল্পের নিজস্ব প্রয়োজন, দ্রুত পরীক্ষা-নিরীক্ষা
Plugin নিজস্ব ফোল্ডার ও manifest	<code>/plugin-name:hello</code>	সহকর্মীদের সঙ্গে ভাগাভাগি, কমিউনিটিতে বিতরণ, version অনুযায়ী রিলিজ, একাধিক প্রকল্পে পুনর্ব্যবহার

টিপস

শুরু করুন `.claude/` দিয়ে — সেখানে দ্রুত পরিবর্তন করে পরীক্ষা করা সহজ। কাজটি স্থির হয়ে গেলে এবং ভাগ করে নেওয়ার প্রয়োজন হলে তখন plugin-এ রূপান্তর করুন। রূপান্তরের পদ্ধতি অধ্যায় ৭-এ বিস্তারিত দেওয়া আছে।

১.৩ Plugin ব্যবহারের সুবিধা

- **একবার সেটআপ, সবখানে ব্যবহার** — user scope-এ ইনস্টল করলে আপনার সব প্রকল্পে চালু থাকে।
- **দলগত সামঞ্জস্য** — project scope-এ ইনস্টল করলে repository-র সব সহযোগী একই টুল পান।
- **Namespace সুরক্ষা** — plugin-এর skill সবসময় `/plugin-name:skill-name` আকারে আসে, তাই দুটি plugin-এ একই নামের skill থাকলেও সংঘর্ষ হয় না।
- **Version নিয়ন্ত্রণ** — `plugin.json` -এ `version` বাড়ালে ব্যবহারকারীরা আপডেট পান; নাহলে পুরোনো সংস্করণেই স্থির থাকেন।
- **এক command-এ ইনস্টল** — MCP server, hook, agent — সব একসঙ্গে কনফিগার হয়ে যায়।

১.৪ খরচের দিকটিও মনে রাখুন: Context cost

প্রতিটি চালু plugin তার skill ও tool-এর বিবরণ Claude-এর **context window**-এ যোগ করে। অর্থাৎ প্রতিটি turn-এ কিছু বাড়তি token খরচ হয়। `/plugin` মেনুর details pane-এ **Context cost** নামে একটি হিসাব দেখানো হয় — ইনস্টলের আগেই বোঝা যায় plugin-টি কত token যোগ করবে।

নোট

দরকারের চেয়ে বেশি plugin চালু রাখলে context ভরে যায় এবং Claude-এর কাজের মান কমতে পারে। যেসব plugin ব্যবহার করছেন না, সেগুলো `/plugin disable` দিয়ে বন্ধ রাখাই ভালো। Claude Code নিজেও অব্যবহৃত plugin-গুলোকে **Not used recently** শিরোনামে আলাদা করে দেখায় — বিস্তারিত অধ্যায় ৬-এ।

Plugin-এর উপাদানসমূহ

একটি plugin আসলে কতগুলো নির্দিষ্ট নামের ফোল্ডার ও ফাইলের সমষ্টি। Claude Code plugin-এর root ডিরেক্টরিতে এই নামগুলো খুঁজে নেয় এবং যা পায় তা-ই লোড করে। নিচে প্রতিটি উপাদানের পরিচয় দেওয়া হলো।

২.১ এক নজরে সব উপাদান

অবস্থান (plugin root-এ)	কাজ
<code>.claude-plugin/plugin.json</code>	Plugin-এর manifest — নাম, version, লেখক ও কনফিগারেশন। উপাদানগুলো ডিফল্ট জায়গায় থাকলে এটি ঐচ্ছিক।
<code>skills/</code>	প্রতিটি skill একটি ফোল্ডার, ভেতরে <code>SKILL.md</code> । Claude প্রসঙ্গ বুঝে নিজেই এগুলো ব্যবহার করে।
<code>commands/</code>	সরল markdown ফাইল, প্রতিটি একেকটি skill। নতুন plugin-এ <code>skills/</code> ব্যবহার করাই ভালো।
<code>agents/</code>	Subagent-এর সংজ্ঞা — নিজস্ব system prompt, model ও tool-সীমা সহ।
<code>hooks/hooks.json</code>	ঘটনানির্ভরিক handler — যেমন প্রতিবার ফাইল লেখার পরে formatter চালানো।
<code>.mcp.json</code>	MCP server-এর কনফিগারেশন, যা বাইরের service-এর tool যোগ করে।
<code>.lsp.json</code>	Language server-এর কনফিগারেশন — code intelligence-এর জন্য।
<code>monitors/monitors.json</code>	পটভূমিতে চলা monitor, যা log বা বাইরের অবস্থা দেখে Claude-কে জানায় (experimental)।
<code>output-styles/</code>	Claude কীভাবে উত্তর দেবে তার ধরন বদলানোর সংজ্ঞা।
<code>themes/</code>	টার্মিনালের রঙের theme (experimental)।
<code>bin/</code>	Executable ফাইল, যেগুলো plugin চালু থাকা অবস্থায় Bash tool-এর <code>PATH</code> -এ যুক্ত হয়।
<code>settings.json</code>	Plugin চালু হলে প্রযোজ্য ডিফল্ট setting। বর্তমানে শুধু <code>agent</code> ও <code>subagentStatusLine</code> সমর্থিত।

সতর্কতা

সবচেয়ে বেশি যে ভুলটি হয়: `commands/`, `agents/`, `skills/` বা `hooks/` ফোল্ডারকে `.claude-plugin/`-এর ভেতরে রাখা। `.claude-plugin/`-এর ভেতরে কেবল `plugin.json` থাকবে; বাকি সব ফোল্ডার plugin root-এ, অর্থাৎ `.claude-plugin/`-এর পাশে থাকতে হবে।

২.২ Skill

Skill হলো plugin-এর সবচেয়ে বেশি ব্যবহৃত উপাদান। প্রতিটি skill একটি ফোল্ডার, যার ভেতরে `SKILL.md` ফাইল থাকে। ফাইলের শুরুতে YAML frontmatter-এ একটি `description` দিতে হয় — এই বর্ণনা দেখেই Claude বুঝতে পারে কখন skill-টি কাজে লাগবে।

skills/code-review/SKILL.md

```
---
description: Reviews code for best practices and potential issues. Use when
  reviewing code, checking PRs, or analyzing code quality.
---
```

```
When reviewing code, check for:
1. Code organization and structure
2. Error handling
3. Security concerns
4. Test coverage
```

Plugin-এর skill সবসময় namespace সহ ডাকা হয় — `/plugin-name:skill-name`। Namespace আসে `plugin.json`-এর `name` ক্ষেত্র থেকে।

২.৩ Agent (Subagent)

Agent হলো বিশেষায়িত সহকারী — নির্দিষ্ট কাজের জন্য আলাদা system prompt, আলাদা model, এমনকি আলাদা tool-সীমা নিয়ে চলে। `agents/` ফোল্ডারে প্রতিটি agent একটি markdown ফাইল:

agents/reviewer.md

```
---
name: code-reviewer
description: Reviews code for quality and security issues
model: sonnet
effort: medium
maxTurns: 20
disallowedTools: Write, Edit
---
```

You are a senior code reviewer with expertise in Python, JavaScript, and TypeScript...

Frontmatter-এ ব্যবহারযোগ্য ক্ষেত্রগুলো: `name`, `description`, `model`, `effort`, `maxTurns`, `tools`, `disallowedTools`, `skills`, `memory`, `background` এবং `isolation`।

২.৪ Hook

Hook হলো ঘটনার সঙ্গে বাঁধা command। উদাহরণস্বরূপ, Claude প্রতিবার কোনো ফাইল `Write` বা `Edit` করার পরে যদি আপনি formatter চালাতে চান:

hooks/hooks.json

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "\\\"${CLAUDE_PLUGIN_ROOT}\\\"/scripts/format.sh"
          }
        ]
      }
    ]
  }
}
```

Hook-এর ধরন হতে পারে `command`, `http`, `mcp_tool`, `prompt` বা `agent`। ঘটনার তালিকায় আছে `SessionStart`, `Setup`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `PostToolBatch`, `Stop`, `SessionEnd` সহ আরও অনেক কিছু।

২.৫ MCP server

MCP (Model Context Protocol) server-এর মাধ্যমে Claude বাইরের service-এর সঙ্গে যুক্ত হয়। Plugin-এ `.mcp.json` ফাইলে server-গুলো ঘোষণা করা হয়:

.mcp.json

```
{
  "mcpServers": {
    "database": {
      "command": "python",
      "args": ["${CLAUDE_PLUGIN_ROOT}/server.py"],
      "env": {
        "DB_PATH": "${CLAUDE_PLUGIN_DATA}/db"
      }
    }
  }
}
```

২.৬ LSP server

LSP server Claude-কে কোড বোঝার ক্ষমতা দেয় — definition-এ যাওয়া, reference খোঁজা এবং সম্পাদনার পরপরই type error দেখা। `.lsp.json` -এ ভাষা অনুযায়ী কনফিগারেশন:

`.lsp.json`

```
{
  "go": {
    "command": "gopls",
    "args": ["serve"],
    "extensionToLanguage": {
      ".go": "go"
    }
  }
}
```

বাধ্যতামূলক ক্ষেত্র দুটি: `command` (binary-টি `PATH`-এ থাকতে হবে) এবং `extensionToLanguage`। ঐচ্ছিক ক্ষেত্রগুলোর মধ্যে আছে `args`, `env`, `settings`, `startupTimeout`, `restartOnCrash` ও `diagnostics`।

২.৭ তিনটি environment variable

Plugin-এর ভেতরের path লেখার সময় এই তিনটি variable ব্যবহার করা যায়। Hook, MCP server ও LSP server-এর subprocess-এ এগুলো পাঠানো হয়:

Variable	যা নির্দেশ করে	কী কাজে
<code>\${CLAUDE_PLUGIN_ROOT}</code>	Plugin ফোল্ডারের পূর্ণ path	সঙ্গে দেওয়া script, binary ও config ফাইল
<code>\${CLAUDE_PLUGIN_DATA}</code>	<code>~/.claude/plugins/data/{id}/</code>	স্থায়ী তথ্য, cache, নির্ভরশীল ফাইল
<code>\${CLAUDE_PROJECT_DIR}</code>	প্রকল্পের root	প্রকল্পের নিজস্ব script

Marketplace কীভাবে কাজ করে

Plugin ইনস্টলের ব্যাপারটা দুই ধাপের। প্রথমে **marketplace** যোগ করতে হয় — এটি কেবল একটি ক্যাটালগ নিবন্ধন করে, কোনো plugin ইনস্টল হয় না। তারপর সেই ক্যাটালগ থেকে আপনি পছন্দমতো plugin ইনস্টল করেন।

৩.১ দুই ধাপের মডেল

- 1 Marketplace যোগ করুন** — এতে ক্যাটালগটি Claude Code-এ নিবন্ধিত হয় এবং আপনি দেখতে পান কী কী পাওয়া যাচ্ছে। এই ধাপে কিছুই ইনস্টল হয় না।
- 2 Plugin ইনস্টল করুন** — ক্যাটালগ থেকে যেটি দরকার সেটি বেছে নিয়ে ইনস্টল করুন।

৩.২ Anthropic-এর তিনটি marketplace

Marketplace	পরিচিতি
claude-plugins-official	Anthropic-এর নিজস্ব বাছাই করা তালিকা। প্রথমবার interactive ভাবে Claude Code চালু করলে এটি স্বয়ংক্রিয়ভাবে যুক্ত হয়ে যায়। যুক্ত না হলে নিজে যোগ করুন: <code>/plugin marketplace add anthropics/claude-plugins-official</code>
claude-community	তৃতীয় পক্ষের plugin, যেগুলো Anthropic-এর স্বয়ংক্রিয় যাচাই ও নিরাপত্তা পরীক্ষায় উত্তীর্ণ হয়েছে। প্রতিটি entry নির্দিষ্ট commit SHA-তে pin করা থাকে। নিজে যোগ করতে হয়: <code>/plugin marketplace add anthropics/claude-plugins-community</code>
claude-code-plugins	উদাহরণ হিসেবে তৈরি demo marketplace, যা plugin দিয়ে কী কী সম্ভব তা দেখায়। যোগ করতে: <code>/plugin marketplace add anthropics/claude-code</code>

নোট

Official marketplace-এ কোন plugin থাকবে তা Anthropic নিজে ঠিক করে; এর জন্য আবেদন করার ব্যবস্থা নেই। অ্যাপের ভেতরের submission form দিয়ে plugin জমা দিলে তা **community** marketplace-এ যায়, official-এ নয়।

৩.৩ Marketplace যোগ করার চারটি উপায়

ক. GitHub repository থেকে

যে repository-তে `.claude-plugin/marketplace.json` ফাইল আছে, তার `owner/repo` লিখলেই হয়:

```
/plugin marketplace add anthropics/claude-code
```

খ. অন্য git host থেকে

পূর্ণ URL দিতে হয়। `.git` suffix লাগবে কি না তা host-ভেদে আলাদা:

Host	নিয়ম
github.com , gitlab.com	<code>.git</code> থাকুক বা না থাকুক — দুটোই চলে।
Azure DevOps	<code>.git</code> বাদ দিন। path-এ <code>/_git/</code> থাকলে Claude Code নিজেই clone করে; suffix জুড়লে clone ব্যর্থ হয়।
অন্য সব host (self-managed GitLab সহ)	<code>.git</code> suffix দিন , নাহলে URL-টিকে সরাসরি <code>marketplace.json</code> -এর লিঙ্ক ধরে নেওয়া হবে।

```
# HTTPS
/plugin marketplace add https://gitlab.com/company/plugins.git

# SSH
/plugin marketplace add git@gitlab.com:company/plugins.git

# নির্দিষ্ট branch বা tag
/plugin marketplace add https://gitlab.com/company/plugins.git#v1.0.0
```

সতর্কতা

URL-এ `https://` অংশটি অবশ্যই লিখুন। শুধু `gitlab.com/company/plugins.git` লিখলে Claude Code v2.1.196 ও পরবর্তী সংস্করণ এটিকে ভুল GitHub `owner/repo` ধরে বাতিল করে দেয়।

গ. স্থানীয় ফোল্ডার থেকে

```
# marketplace.json আছে এমন ফোল্ডার
/plugin marketplace add ./my-marketplace

# অথবা সরাসরি ফাইলের path
/plugin marketplace add ./path/to/marketplace.json
```

ঘ. Remote URL থেকে

```
/plugin marketplace add https://example.com/marketplace.json
```

টিপস

সংক্ষেপে লিখতে চাইলে `/plugin marketplace` -এর বদলে `/plugin market` এবং `remove` -এর বদলে `rm` ব্যবহার করা যায়।

অত্যাবশ্যকীয় Plugin-এর তালিকা

Official marketplace `claude-plugins-official` -এ plugin-গুলো কয়েকটি শ্রেণিতে ভাগ করা। নিচে প্রতিটি শ্রেণির plugin, তার কাজ এবং প্রয়োজনীয় পূর্বশর্ত দেওয়া হলো। ইনস্টলের সাধারণ নিয়ম:

```
/plugin install <plugin-name>@claude-plugins-official
```

8.১ Code intelligence (LSP) plugin

এই plugin-গুলো Claude Code-এর ভেতরের LSP tool চালু করে। ফলে Claude definition-এ যেতে পারে, reference খুঁজতে পারে, এবং প্রতিটি সম্পাদনার পরপরই type error দেখতে পায়। এতে দুটি বড় সুবিধা:

- **স্বয়ংক্রিয় diagnostics** — Claude ফাইল সম্পাদনা করার পরপরই language server error ও warning জানিয়ে দেয়। compiler বা linter আলাদা করে চালাতে হয় না; ভুল করলে Claude সেই turn-এই সেটি ঠিক করে নেয়।
- **Code navigation** — definition, reference, hover type, symbol তালিকা, implementation ও call hierarchy — grep-ভিত্তিক খোঁজার চেয়ে অনেক নির্ভুল।

ভাষা	Plugin	যে binary আগে ইনস্টল করতে হবে
TypeScript	<code>typescript-lsp</code>	<code>typescript-language-server</code>
Python	<code>pyright-lsp</code>	<code>pyright-langserver</code>
Rust	<code>rust-analyzer-lsp</code>	<code>rust-analyzer</code>
Go	<code>gopls-lsp</code>	<code>gopls</code>
C / C++	<code>clangd-lsp</code>	<code>clangd</code>
C#	<code>csharp-lsp</code>	<code>csharp-ls</code>
Java	<code>jdtls-lsp</code>	<code>jdtls</code>
Kotlin	<code>kotlin-lsp</code>	<code>kotlin-language-server</code>
Lua	<code>lua-lsp</code>	<code>lua-language-server</code>
PHP	<code>php-lsp</code>	<code>intelephense</code>
Swift	<code>swift-lsp</code>	<code>sourcekit-lsp</code>

সতর্কতা

Plugin-টি language server-এর binary **নিজে ইনস্টল করে না** — সেটি আপনাকে আলাদাভাবে ইনস্টল করতে হবে। ইনস্টলের পরে `/plugin` -এর **Errors** ট্যাবে `Executable not found in $PATH` দেখলে বুঝবেন binary-টি অনুপস্থিত। এছাড়া cloud session-এ Claude Code plugin-এর language server চালু করে না, তাই সেখানে LSP tool পাওয়া যায় না।

8.২ External integration — বাইরের service-এর সংযোগ

এই plugin-গুলোর সঙ্গে আগে থেকেই কনফিগার করা MCP server দেওয়া থাকে, ফলে হাতে হাতে সেটআপ ছাড়াই Claude-কে বাইরের service-এর সঙ্গে যুক্ত করা যায়।

শ্রেণি	Plugin	কাজ
Source control	github	Pull request, issue, review comment, CI-র অবস্থা — সরাসরি Claude থেকে দেখা ও পরিচালনা
	gitlab	GitLab-এর merge request, issue ও pipeline-এর সঙ্গে সংযোগ
Project management	atlassian	Jira ও Confluence — ticket পড়া, আপডেট করা, ডকুমেন্ট খোঁজা
	asana	Asana-র task ও project-এর সঙ্গে কাজ
	linear	Linear-এর issue তৈরি, খোঁজা ও অবস্থা পরিবর্তন
	notion	Notion-এর page ও database পড়া ও লেখা
Design	figma	Figma-র ডিজাইন থেকে কোড তৈরি এবং কোড থেকে ডিজাইন তৈরি
Infrastructure	vercel	Deployment, build log, runtime error ও project ব্যবস্থাপনা
	firebase	Firebase-এর প্রকল্প ও service-এর সঙ্গে সংযোগ
	supabase	Supabase-এর database ও backend পরিচালনা
Communication	slack	Slack-এর চ্যানেল ও বার্তার সঙ্গে কাজ
Monitoring	sentry	Sentry-তে ধরা পড়া error ও stack trace বিশ্লেষণ

8.৩ নিরাপত্তা

Plugin	কাজ
security-guidance	Claude-এর করা প্রতিটি পরিবর্তন পরীক্ষা করে সাধারণ নিরাপত্তা-দুর্বলতা খোঁজে এবং যা পায় তা একই session-এই ঠিক করার নির্দেশ দেয়। প্রকল্পভিত্তিক নিজস্ব নিয়মও যোগ করা যায়।

8.8 Development workflow

Plugin	কাজ
<code>commit-commands</code>	Git-এর কাজের skill — commit করা, push করা এবং pull request তৈরি। যেমন <code>/commit-commands:commit</code> চালালে পরিবর্তন stage হয়, commit message তৈরি হয় এবং commit হয়ে যায়।
<code>pr-review-toolkit</code>	Pull request পর্যালোচনার জন্য বিশেষায়িত agent-এর সংগ্রহ।
<code>agent-sdk-dev</code>	Claude Agent SDK দিয়ে কিছু বানাতে গেলে প্রয়োজনীয় টুল ও নির্দেশনা।
<code>plugin-dev</code>	নিজের plugin তৈরির পূর্ণ toolkit — এই বইয়ের অধ্যায় ৭ পড়ার পাশাপাশি এটি ইনস্টল করে নিলে কাজ অনেক সহজ হয়।

8.৫ Output style

Plugin	কাজ
<code>explanatory-output-style</code>	Claude কেন কোন সিদ্ধান্ত নিল — বাস্তবায়নের পেছনের যুক্তি ব্যাখ্যা করে বলে।
<code>learning-output-style</code>	শেখার উপযোগী interactive ধরন, যেখানে কাজ করতে করতে দক্ষতা তৈরি হয়।

8.৬ কোনগুলো দিয়ে শুরু করবেন

সব plugin একসঙ্গে ইনস্টল করার দরকার নেই — প্রতিটি context-এর খরচ বাড়ায়। একজন সাধারণ ডেভেলপারের জন্য নিচের চারটি দিয়ে শুরু করাই যথেষ্ট:

ইনস্টল করুন	কারণ
<code><আপনার ভাষার>-lsp</code>	সবচেয়ে বেশি ফল দেয় — Claude-এর ভুল সঙ্গে সঙ্গে ধরা পড়ে
<code>github</code> বা <code>gitlab</code>	PR ও issue নিয়ে কাজ টার্মিনাল ছাড়াই করা যায়
<code>commit-commands</code>	প্রতিদিনের git-এর কাজ দ্রুত হয়
<code>security-guidance</code>	নিরাপত্তার ভুল কোডে ঢোকার আগেই ধরা পড়ে

প্রয়োজন বুঝে পরে `linear`, `notion`, `sentry` বা `figma` -র মতো integration যোগ করে নিতে পারেন। পুরো ক্যাটালগ দেখতে `/plugin` চালিয়ে **Discover** ট্যাবে যান, অথবা `claude.com/plugins` ঠিকানায় দেখুন।

ধাপে ধাপে Plugin সংযোজন

এই অধ্যায়ে তিনটি আলাদা পরিবেশের জন্য পূর্ণাঙ্গ নির্দেশনা দেওয়া হয়েছে — টার্মিনালের **CLI**, **Desktop app** এবং **Web / Cloud session**। আপনি যেটি ব্যবহার করেন, সেই অংশটি অনুসরণ করুন।

৫.১ Claude Code CLI (টার্মিনাল)

এটিই সবচেয়ে পূর্ণাঙ্গ পথ — `/plugin` command এখানেই পাওয়া যায়। উদাহরণ হিসেবে আমরা demo marketplace থেকে `commit-commands` plugin-টি ইনস্টল করব।

1 Marketplace যোগ করুন। Claude Code চালু অবস্থায় লিখুন:

```
/plugin marketplace add anthropics/claude-code
```

এতে ক্যাটালগটি ডাউনলোড হয়ে তার plugin-গুলো আপনার জন্য উন্মুক্ত হয়। Official marketplace-টি সাধারণত আগে থেকেই যোগ করা থাকে, তাই সেখান থেকে ইনস্টল করলে এই ধাপটি লাগে না।

2 Plugin manager খুলুন। লিখুন:

```
/plugin
```

চারটি ট্যাব আসবে; **Tab** চেপে সামনে আর **Shift+Tab** চেপে পেছনে যাওয়া যায়:

Discover	আপনার যোগ করা সব marketplace-এর plugin দেখা
Installed	ইনস্টল করা plugin দেখা ও পরিচালনা
Marketplaces	Marketplace যোগ, বাদ ও হালনাগাদ করা
Errors	Plugin লোড হতে গিয়ে হওয়া ত্রুটি দেখা

3 Plugin বেছে নিয়ে বিবরণ দেখুন। **Discover** ট্যাবে গিয়ে একটি plugin নির্বাচন করলে details pane-এ যা দেখবেন:

- **Context cost** — প্রতিটি turn-এ কত token যোগ হবে তার হিসাব
- **Last updated** — সর্বশেষ কবে হালনাগাদ হয়েছে
- **Will install** — ঠিক কোন কোন command, agent, skill, hook, MCP ও LSP server যুক্ত হবে তার তালিকা

সব plugin এই তথ্য সরবরাহ করে না। স্থানীয় বা custom marketplace-এর ক্ষেত্রে Context cost ও Last updated না-ও দেখা যেতে পারে, এবং Will install-এর জায়গায় *Components will be discovered at installation* লেখা থাকতে পারে।

4 **Scope নির্বাচন করুন।** ইনস্টলের সময় তিনটি বিকল্প আসে:

User	শুধু আপনার জন্য, কিন্তু সব প্রকল্পে
Project	এই repository-র সব সহযোগীর জন্য (<code>.claude/settings.json</code> -এ যুক্ত হয়)
Local	শুধু আপনার জন্য, শুধু এই repository-তে

5 **ইনস্টল করুন।** মেনু থেকে নির্বাচন করে **Enter** চাপুন, অথবা সরাসরি command লিখুন:

```
/plugin install commit-commands@claude-code-plugins
```

সাধারণ গঠনটি হলো:

```
/plugin install <plugin-name>@<marketplace-name>
```

6 **ইনস্টলের সারসংক্ষেপ পড়ুন।** দুটি বার্তার যেকোনো একটি আসবে:

Plugin is now active.	কিছু করার নেই, plugin চালু হয়ে গেছে
Run <code>/reload-plugins</code> to activate.	নিচের ধাপটি চালাতে হবে

```
/reload-plugins
```

Reload করলে prompt cache বাতিল হয়ে যেতে পারে — সেক্ষেত্রে Claude Code সতর্ক করে থেমে যায়। তখন `/reload-plugins --force` দিয়ে আবার চালান।

7 **Plugin ব্যবহার করুন।** Plugin-এর skill namespace সহ ডাকতে হয়:

```
/commit-commands:commit
```

এতে আপনার পরিবর্তন stage হবে, একটি commit message তৈরি হবে এবং commit হয়ে যাবে। কোন plugin কী কী skill দেয় তা **Discover** ট্যাবের বিবরণে অথবা plugin-এর homepage-এ পাওয়া যায়।

Interactive ধাপ ছাড়া ইনস্টল

Script-এ ব্যবহারের জন্য shell command রয়েছে, যা কোনো মেনু খোলে না। এটি ডিফল্টভাবে user scope-এ ইনস্টল করে:

```
claude plugin install formatter@your-org --scope project
claude plugin uninstall formatter@your-org --scope project
```

নোট

`claude plugin install` চলমান session-এর ভেতরে চলে না, তাই ইনস্টল করা plugin পরের বার Claude Code চালু করলে লোড হবে — অথবা খোলা session-এ `/reload-plugins` চালালে।

৫.২ Claude Desktop app

Desktop app-এ টার্মিনাল ছাড়াই plugin ইনস্টল করা যায়। App-এর তিনটি ট্যাবের মধ্যে **Code** ট্যাবটি সফটওয়্যার ডেভেলপমেন্টের জন্য।

- 1 **Plugin মেনু খুলুন।** Prompt box-এর পাশে + বোতামে ক্লিক করে **Plugins** নির্বাচন করুন। এখানে ইনস্টল করা plugin ও তাদের skill-এর তালিকা দেখা যায়।
- 2 **Plugin browser খুলুন।** ওই submenu থেকে **Add plugin** বেছে নিন। Plugin browser খুলবে, যেখানে আপনার কনফিগার করা সব marketplace-এর — official Anthropic marketplace সহ — plugin দেখা যাবে।
- 3 **Scope বেছে ইনস্টল করুন।** CLI-র মতোই user account, নির্দিষ্ট project, অথবা local — এই তিনটি scope-এর যেকোনো একটিতে ইনস্টল করা যায়।
- 4 **পরিচালনা করুন।** Plugin চালু, বন্ধ বা মুছে ফেলতে submenu থেকে **Manage plugins** নির্বাচন করুন।
- 5 **এক জায়গায় সবকিছু।** Connector, skill ও plugin একসঙ্গে পরিচালনা করতে sidebar-এর **Customize**-এ ক্লিক করুন।

Desktop app-এর **Cowork** ট্যাবটি তার skill, plugin ও connector এই Customize কনফিগারেশন থেকেই নেয় — CLI-র `~/ .claude` ডিরেক্টরি থেকে নয়। এটি আপনার claude.ai অ্যাকাউন্টের মাধ্যমে sync হয়।

সতর্কতা — Desktop app-এর সীমাবদ্ধতা

Plugin browser **cloud session-এ পাওয়া যায় না**, এবং desktop app থেকে ইনস্টল করা plugin cloud session-এ কাজ করে না। এছাড়া **WSL session-এ** plugin ব্যবহার করা যায় না। Cloud session-এ plugin দরকার হলে পরের অংশটি দেখুন।

Plugin browser স্থানীয় ও SSH session-এ কাজ করে। SSH-এর ক্ষেত্রে দূরবর্তী মেশিনটি Linux বা macOS হতে হবে; প্রথমবার সংযোগ করলে Desktop নিজেই সেখানে Claude Code ইনস্টল করে নেয়।

৫.৩ Web ও Cloud session

Claude Code যখন claude.ai-তে বা cloud environment-এ চলে, তখন `/plugin` মেনু থাকে না। সেখানে plugin যোগ করার দুটি পথ আছে।

পথ ১ — repository-র settings ফাইলে ঘোষণা

প্রকল্পের `.claude/settings.json` ফাইলে plugin ঘোষণা করলে Claude Code session শুরুর সময়ই সেটি ইনস্টল করে নেয়। Marketplace-টিও একই ফাইলে নিবন্ধন করতে হবে:

`.claude/settings.json`

```
{
  "extraKnownMarketplaces": {
    "my-team-tools": {
      "source": {
        "source": "github",
        "repo": "your-org/claude-plugins"
      }
    }
  },
  "enabledPlugins": {
    "formatter@my-team-tools": true,
    "legacy-helper@my-team-tools": false
  }
}
```

`enabledPlugins` -এ প্রতিটি plugin-এর পরিচয় `plugin-name@marketplace-name` আকারে লিখে `true` বা `false` দিতে হয়। এই ফাইলটি version control-এ থাকায় দলের সবাই একই কনফিগারেশন পান।

পথ ২ — claude.ai অ্যাকাউন্টে চালু করা

আপনার claude.ai অ্যাকাউন্টে plugin-টি চালু করে রাখলে Claude Code সেটিকে **synced plugin** হিসেবে লোড করে। এতে repository-তে কিছু যোগ না করেও cloud session-এ plugin পাওয়া যায়।

৫.৪ তিনটি পরিবেশের তুলনা

বৈশিষ্ট্য	CLI	Desktop	Web / Cloud
<code>/plugin</code> মেনু	আছে	Plugin browser	নেই
Marketplace যোগ করা	Command দিয়ে	UI দিয়ে	<code>extraKnownMarketplaces</code>
Plugin ইনস্টল	<code>/plugin install</code>	Add plugin	<code>enabledPlugins</code> বা <code>synced</code>
Scope নির্বাচন	তিনটিই	তিনটিই	Repository-নির্ভর
LSP server চালু হয়	হ্যাঁ	হ্যাঁ	না
Session-এ reload	<code>/reload-plugins</code>	<code>/reload-plugins</code>	Session শুরুতে

ব্যবস্থাপনা, Scope ও হালনাগাদ

ইনস্টল করার পরের কাজ — কোন plugin কোথায় চালু থাকবে, কোনটি বন্ধ রাখবেন, আর কখন হালনাগাদ হবে তা ঠিক করা।

৬.১ চারটি scope

Scope	Settings ফাইল	কোথায় কার্যকর
user	~/.claude/settings.json	আপনার সব প্রকল্পে (ডিফল্ট)
project	.claude/settings.json	Version control-এর মাধ্যমে পুরো দলের জন্য
local	.claude/settings.local.json	শুধু এই প্রকল্পে, শুধু আপনার জন্য (gitignore করা)
managed	Managed settings	প্রতিষ্ঠানের প্রশাসক নিয়ন্ত্রিত — ব্যবহারকারী বদলাতে পারেন না

৬.২ দৈনন্দিন command

```
# ইনস্টল করা plugin-এর তালিকা
/plugin list
/plugin list --enabled
/plugin list --disabled

# একটি plugin-এর বিস্তারিত
/plugin info plugin-name

# না মুছে সাময়িকভাবে বন্ধ রাখা
/plugin disable plugin-name@marketplace-name

# আবার চালু করা
/plugin enable plugin-name@marketplace-name

# সম্পূর্ণ মুছে ফেলা
/plugin uninstall plugin-name@marketplace-name
```

নোট

`/plugin disable`, `/plugin enable` ও `/plugin uninstall` চালালে Claude Code plugin panel খুলে পরিবর্তনটি প্রয়োগ করে এবং panel খোলা রেখে দেয়। পরের command লেখার আগে **Esc** চেপে panel বন্ধ করুন। Script-এর জন্য `claude plugin` shell command ব্যবহার করুন — সেগুলো panel খোলে না।

উপরের পরিচয়গুলোতে `plugin-name` বলতে **marketplace entry-র** নাম বোঝায়, যা plugin-এর নিজের `plugin.json`-এর `name` থেকে আলাদা হতে পারে।

৬.৩ Installed ট্যাব

`/plugin` চালিয়ে **Installed** ট্যাবে গেলে তালিকাটি scope অনুযায়ী ভাগ করা থাকে এবং সমস্যাগুলো আগে দেখানো হয় — লোড হতে ব্যর্থ বা নির্ভরতা অমীমাংসিত plugin সবার উপরে, তারপর আপনার প্রিয় (favorite) plugin, আর বন্ধ করা plugin সবার নিচে একটি ভাঁজ করা শিরোনামের আড়ালে।

- `f` চেপে কোনো plugin favorite করা বা বাদ দেওয়া যায়
- টাইপ করে নাম বা বর্ণনা দিয়ে ছেকে নেওয়া যায়
- **Enter** চেপে বিস্তারিত দেখে সেখান থেকেই enable, disable বা uninstall করা যায়

Not used recently

আপনি নিজে ইনস্টল করেছেন কিন্তু অন্তত দুই সপ্তাহ ও অন্তত ১০টি session ধরে ব্যবহার করেননি — এমন plugin **Not used recently** শিরোনামে আলাদা দেখানো হয়, সঙ্গে **Last used** তারিখ। যেগুলো আর কাজে লাগছে না অথচ প্রতিটি session-এ context ও startup খরচ বাড়চ্ছে, সেগুলো এখান থেকে খুঁজে বন্ধ বা মুছে ফেলুন।

দুই ধরনের plugin কখনোই অব্যবহৃত হিসেবে দেখানো হয় না:

- প্রতিষ্ঠান-নিয়ন্ত্রিত plugin এবং `--plugin-dir` দিয়ে লোড করা plugin
- যেসব plugin theme, output style, monitor বা workflow দেয় — কারণ এগুলো আলাদা করে ডাকতে হয় না

৬.৪ Marketplace ব্যবস্থাপনা

```
# যোগ করা সব marketplace-এর তালিকা
/plugin marketplace list

# ক্যাটালগ হালনাগাদ করা
/plugin marketplace update marketplace-name

# marketplace বাদ দেওয়া
/plugin marketplace remove marketplace-name
```

সতর্কতা

কোনো marketplace বাদ দিলে সেখান থেকে ইনস্টল করা সব plugin মুছে যাবে।

৬.৫ স্বয়ংক্রিয় হালনাগাদ

Session শুরুর পরে Claude Code পটভূমিতে marketplace ও ইনস্টল করা plugin হালনাগাদ করতে পারে। চলমান session যেন হঠাৎ বদলে না যায়, সেজন্য এই পরীক্ষা **১০ মিনিট পর্যন্ত এলোমেলো বিলম্বে** চালানো হয়। কিছু হালনাগাদ হলে `/reload-plugins` চালানোর পরামর্শ দেখানো হয়; নাহলে পরের বার চালু করার সময় নতুন সংস্করণ লোড হয়।

UI থেকে প্রতিটি marketplace-এর জন্য আলাদাভাবে চালু বা বন্ধ করা যায়:

- 1 `/plugin` চালিয়ে plugin manager খুলুন
- 2 **Marketplaces** নির্বাচন করুন
- 3 তালিকা থেকে একটি marketplace বাছুন
- 4 **Enable auto-update** বা **Disable auto-update** বেছে নিন

`claude-plugins-official` এবং Anthropic-এর অন্যান্য official marketplace-এ auto-update ডিফল্টভাবে চালু থাকে। তৃতীয় পক্ষের ও স্থানীয় development marketplace-এ ডিফল্টভাবে বন্ধ থাকে।

পরিবেশ-চলক দিয়ে নিয়ন্ত্রণ:

```
# Claude Code ও plugin – দুটোরই auto-update বন্ধ
export DISABLE_AUTOUPDATER=1

# Claude Code-এর auto-update বন্ধ, কিন্তু plugin-এর চালু
export DISABLE_AUTOUPDATER=1
export FORCE_AUTOUPDATE_PLUGINS=1
```

৬.৬ ইনস্টলের সময় ক্যাটালগ হালনাগাদের নিয়ম

Plugin-এর নাম কীভাবে লিখছেন তার উপর নির্ভর করে Claude Code ক্যাটালগ আগে হালনাগাদ করবে কি না:

যেভাবে লিখলেন	যা ঘটে
<code>plugin-name@marketplace-name</code>	খোঁজার আগে ওই marketplace হালনাগাদ করা হয় — auto-update বন্ধ থাকলেও। হালনাগাদ ব্যর্থ হলে (যেমন ইন্টারনেট না থাকলে) cache করা ক্যাটালগ থেকেই খোঁজা হয়।
শুধু <code>plugin-name</code>	প্রথমে cache-এ খোঁজা হয়; না পেলে কেবল সেই marketplace-গুলো হালনাগাদ হয় যেগুলোর auto-update চালু। নতুন প্রকাশিত plugin ইনস্টল করতে আগে <code>/plugin marketplace update <name></code> চালিয়ে নিন।

৬.৭ Reload-এর খরচ

`/reload-plugins` চালালে Claude Code সব সক্রিয় plugin আবার লোড করে এবং plugin, skill, agent, hook, MCP server ও LSP server-এর সংখ্যা দেখায়। নতুন লোড হওয়া উপাদানগুলো conversation-এ নিজেদের পরিচয় যোগ করে, ফলে পরের request-এ কিছু বাড়তি token খরচ হয়। বিশেষ করে যেসব plugin MCP server দেয়, সেগুলোর ক্ষেত্রে খরচ বেশি — পুরো conversation আবার পড়া লাগতে পারে।

নিজের Plugin তৈরি

এবার উল্টো দিক — ব্যবহারকারী থেকে নির্মাতা। এই অধ্যায়ে শূন্য থেকে একটি plugin তৈরি করে পরীক্ষা করা, তারপর তাতে agent, hook ও MCP server যোগ করা শেখানো হয়েছে।

৭.১ প্রথম Plugin — হাতেকলমে

শুরুর আগে Claude Code ইনস্টল ও authenticate করা থাকতে হবে।

- 1 Plugin-এর ফোল্ডার তৈরি করুন।** জায়গাটা এখন গুরুত্বপূর্ণ নয়, কারণ পরীক্ষার সময় আমরা `--plugin-dir` দিয়ে সরাসরি এই ফোল্ডারটিই দেখিয়ে দেব:

```
mkdir my-first-plugin
```

- 2 Manifest লিখুন।** প্রথমে `.claude-plugin` ফোল্ডার:

```
mkdir my-first-plugin/.claude-plugin
```

তারপর তার ভেতরে `plugin.json`:

my-first-plugin/.claude-plugin/plugin.json

```
{
  "name": "my-first-plugin",
  "description": "A greeting plugin to learn the basics",
  "version": "1.0.0",
  "author": {
    "name": "Your Name"
  }
}
```

ক্ষেত্র	কাজ
name	অনন্য পরিচয় এবং skill-এর namespace — skill হবে <code>/my-first-plugin:hello</code>
description	Plugin manager-এ যা দেখা যাবে
version	ঐচ্ছিক। দিলে এটি না বাড়ানো পর্যন্ত ব্যবহারকারীরা হালনাগাদ পাবেন না
author	ঐচ্ছিক, কৃতিত্বের জন্য

- 3 একটি skill যোগ করুন। Skill থাকে `skills/` ফোল্ডারে; ফোল্ডারের নামই skill-এর নাম হয়:

```
mkdir -p my-first-plugin/skills/hello
```

my-first-plugin/skills/hello/SKILL.md

```
---
description: Greet the user with a friendly message
disable-model-invocation: true
---

Greet the user warmly and ask how you can help them today.
```

- 4 পরীক্ষা করুন। `--plugin-dir` দিয়ে Claude Code চালু করলে plugin-টি ইনস্টল ছাড়াই লোড হয়:

```
claude --plugin-dir ./my-first-plugin
```

চালু হওয়ার পরে skill-টি ডাকুন:

```
/my-first-plugin:hello
```

`/help` চালিয়ে **Custom commands** ট্যাবে গেলে আপনার skill-টি plugin namespace-এর নিচে তালিকাভুক্ত দেখতে পাবেন।

- 5 Argument যোগ করুন। `$ARGUMENTS` লিখলে skill-এর নামের পরে ব্যবহারকারীর লেখা যেকোনো টেক্সট সেখানে বসে যায়:

my-first-plugin/skills/hello/SKILL.md

```
---
description: Greet the user with a personalized message
---

# Hello Skill

Greet the user named "$ARGUMENTS" warmly and ask how you can help
them today. Make the greeting personal and encouraging.
```

পরিবর্তন কার্যকর করতে `/reload-plugins` চালিয়ে তারপর:

```
/my-first-plugin:hello Alex
```

নোট — Namespace কেন?

Plugin-এর skill সবসময় namespace সহ থাকে (`/my-first-plugin:hello`), যাতে দুটি ভিন্ন plugin-এ একই নামের skill থাকলেও সংঘর্ষ না হয়। Namespace বদলাতে চাইলে `plugin.json` -এর `name` ক্ষেত্রটি বদলান।

৭.২ Skills ডিরেক্টরিতে plugin রাখা

প্রতিবার `--plugin-dir` লিখতে না চাইলে plugin-টি সরাসরি আপনার skills ডিরেক্টরিতে রাখতে পারেন — তখন Claude Code নিজেই সেটি লোড করে নেয়। `claude plugin init` কাঠামোটি তৈরি করে দেয়:

```
claude plugin init my-tool

# নির্দিষ্ট উপাদান সহ
claude plugin init my-tool --with skills agents hooks mcp
```

এতে `~/.claude/skills/my-tool/` তৈরি হয়, ভেতরে `.claude-plugin/plugin.json` ও একটি প্রাথমিক `SKILL.md` থাকে। পরের session-এ এটি `my-tool@skills-dir` হিসেবে লোড হয় — কোনো marketplace বা ইনস্টলের ধাপ লাগে না।

৭.৩ পূর্ণাঙ্গ ডিরেক্টরি কাঠামো

```
plugin-name/
├── .claude-plugin/
│   └── plugin.json          # manifest (ঐচ্ছিক)
├── skills/                  # SKILL.md সহ ফোল্ডার
│   └── skill-name/
│       └── SKILL.md
├── commands/               # সরল .md ফাইল
├── agents/                  # subagent-এর সংজ্ঞা
├── workflows/
├── output-styles/
├── themes/                  # experimental
├── monitors/                # experimental
├── hooks/
│   └── hooks.json
├── .mcp.json
├── .lsp.json
├── bin/                     # PATH-এ যুক্ত হয়
├── scripts/
├── package.json
└── LICENSE
```

Plugin-এ একটিমাত্র skill থাকলে `skills/` ফোল্ডার না বানিয়ে সরাসরি plugin root-এ `SKILL.md` রাখা যায়; তখন frontmatter-এর `name` ক্ষেত্রটি skill-এর নাম হিসেবে ব্যবহৃত হয়। ভবিষ্যতে একাধিক skill যোগ করার সম্ভাবনা থাকলে `skills/` কাঠামোই ব্যবহার করুন।

৭.৪ plugin.json — পূর্ণ schema

```
{
  "name": "plugin-name",
  "displayName": "Human Readable Name",
  "version": "1.0.0",
  "description": "What this plugin does",
  "author": { "name": "Author Name", "email": "author@example.com" },
  "homepage": "https://docs.example.com/plugin",
  "repository": "https://github.com/author/plugin",
  "license": "MIT",
  "keywords": ["keyword1", "keyword2"],
  "defaultEnabled": true,

  "skills": "./custom/skills/",
  "commands": [". /commands/", ". /extra.md"],
  "agents": "./custom/agents/",
  "hooks": "./hooks/hooks.json",
  "mcpServers": "./mcp-config.json",
  "lspServers": "./.lsp.json",

  "userConfig": {
    "api_token": {
      "type": "string",
      "title": "API Token",
      "description": "Authentication token",
      "sensitive": true,
      "required": true
    }
  },

  "dependencies": [
    "helper-plugin",
    { "name": "secrets-vault", "version": "~2.1.0" }
  ]
}
```

বাধ্যতামূলক ক্ষেত্র কেবল একটি — `name`, যা kebab-case-এ লিখতে হয়।

সতর্কতা — path-এর নিয়ম

সব path আপেক্ষিক হতে হবে এবং `./` দিয়ে শুরু করতে হবে। আরেকটি সূক্ষ্ম পার্থক্য মনে রাখুন: `skills` ক্ষেত্রটি ডিফল্ট `skills/` স্ক্যানের সঙ্গে **যোগ হয়**, কিন্তু `commands`, `agents`, `workflows` ও `outputStyles` ডিফল্ট অবস্থানকে **প্রতিস্থাপন করে**।

userConfig — ব্যবহারকারীর কাছ থেকে তথ্য নেওয়া

`userConfig`-এ ঘোষিত মান LSP ও MCP কনফিগারেশনে এবং skill-এর ভেতরে `${user_config.database_url}` আকারে ব্যবহার করা যায়। গোপনীয় তথ্যের জন্য `"sensitive": true` দিন।

৭.৫ পরীক্ষা ও ত্রুটি নির্ণয়

```
# স্থানীয় ফোল্ডার থেকে লোড
claude --plugin-dir ./my-plugin

# .zip আর্কাইভ থেকেও চলে
claude --plugin-dir ./my-plugin.zip

# একাধিক plugin একসঙ্গে
claude --plugin-dir ./plugin-one --plugin-dir ./plugin-two

# URL-এ রাখা .zip থেকে (শুধু ওই session-এর জন্য)
claude --plugin-url https://example.com/my-plugin.zip
```

`--plugin-dir` দিয়ে লোড করা plugin-এর নাম যদি ইনস্টল করা কোনো marketplace plugin-এর নামের সঙ্গে মিলে যায়, তবে ওই session-এ স্থানীয় কপিটিই অগ্রাধিকার পায়। ফলে ইনস্টল করা plugin আনইনস্টল না করেই তার পরিবর্তন পরীক্ষা করা যায়।

জমা দেওয়ার আগে বা বিতরণের আগে যাচাই করে নিন:

```
claude plugin validate ./my-plugin
claude plugin validate ./my-plugin --strict
```

সফল হলে `✓ Validation passed` লেখা আসে; warning থাকলে `✓ Validation passed with warnings`। Warning সাধারণত যাচাই ব্যর্থ করে না, তবে `--strict` দিলে warning-ও ত্রুটি হিসেবে গণ্য হয়।

সমস্যা হলে যা দেখবেন:

- **কাঠামো ঠিক আছে কি না** — ফোল্ডারগুলো plugin root-এ আছে তো, `.claude-plugin/` -এর ভেতরে নয়?
- **প্রতিটি উপাদান আলাদা করে** — skill, agent ও hook একে একে পরীক্ষা করুন
- **Skill** — `/plugin-name:skill-name` দিয়ে ডেকে দেখুন
- **Agent** — `/context` -এর **Custom Agents** অংশে আছে কি না দেখুন, বা নাম ধরে @-mention করুন
- **Hook** — যে ঘটনার সঙ্গে বাঁধা সেটি ঘটান (যেমন `PostToolUse` -এর জন্য Claude-কে একটি ফাইল সম্পাদনা করতে বলুন) এবং debug log-এ ফলাফল দেখুন
- **ত্রুটি** — `/plugin` -এর **Errors** ট্যাব, আর বিস্তারিত জানতে `claude --debug`

৭.৬ পুরোনো কনফিগারেশনকে plugin-এ রূপান্তর

`.claude/` ফোল্ডারে আগে থেকেই skill বা hook থাকলে সেগুলো plugin-এ নিয়ে আসা সহজ:

1 Plugin-এর কাঠামো ও manifest তৈরি করুন:

```
mkdir -p my-plugin/.claude-plugin
```

2 বিদ্যমান ফোল্ডারগুলো কপি করুন (যেটি নেই, সেই command বাদ দিন):

```
cp -r .claude/commands my-plugin/  
cp -r .claude/agents my-plugin/  
cp -r .claude/skills my-plugin/
```

3 Hook থাকলে `settings.json` -এর `hooks` অংশটি হুবহু `my-plugin/hooks/hooks.json` -এ নিয়ে যান — গঠন একই।

4 পরীক্ষা করুন:

```
claude --plugin-dir ./my-plugin
```

Standalone (<code>.claude/</code>)	Plugin
শুধু একটি প্রকল্পে পাওয়া যায়	Marketplace দিয়ে ভাগ করা যায়
ফাইল থাকে <code>.claude/commands/</code> -এ	ফাইল থাকে <code>plugin-name/commands/</code> -এ
Hook থাকে <code>settings.json</code> -এ	Hook থাকে <code>hooks/hooks.json</code> -এ
ভাগ করতে হাতে কপি করতে হয়	<code>/plugin install</code> দিয়েই হয়ে যায়

নোট

রূপান্তরের পরে `.claude/` -এর মূল ফাইলগুলো মুছে ফেলুন, নাহলে একই জিনিস দুবার থাকবে। বিশেষ করে `project` ও `user` স্তরের `.claude/agents/` -এর সংজ্ঞা একই নামের plugin agent-কে ছাপিয়ে যায় — মূল ফাইল না সরালে plugin-এর সংস্করণটি কার্যকর হবে না।

দলগত Marketplace ও নিরাপত্তা

সতর্কতা — সবার আগে পড়ুন

Plugin ও marketplace অত্যন্ত **বিশ্বাসনির্ভর** উপাদান — এগুলো আপনার মেশিনে আপনারই অধিকার নিয়ে যেকোনো কোড চালাতে পারে। Anthropic নিয়ন্ত্রণ করে না কোনো plugin-এর ভেতরে কী MCP server, ফাইল বা সফটওয়্যার আছে, এবং সেগুলো ঠিকঠাক কাজ করে কি না তারও নিশ্চয়তা দেয় না। **কেবল বিশ্বস্ত উৎস থেকে** plugin ইনস্টল করুন এবং marketplace যোগ করুন। ইনস্টলের আগে plugin-এর homepage দেখে নিন।

৮.১ দলের জন্য marketplace কনফিগার করা

প্রকল্পের `.claude/settings.json`-এ marketplace ঘোষণা করে রাখলে দলের সদস্যরা আলাদা করে যোগ করার ঝামেলা থেকে বাঁচেন। সদস্য একবার repository-র folder-টিকে **trust** করলে Claude Code আর কিছু জিজ্ঞেস না করেই marketplace-গুলো যোগ করে নেয়।

`.claude/settings.json`

```
{
  "extraKnownMarketplaces": {
    "my-team-tools": {
      "source": {
        "source": "github",
        "repo": "your-org/claude-plugins"
      }
    }
  }
}
```

নোট

Marketplace যোগ হওয়া মানেই plugin ইনস্টল হয়ে যাওয়া নয়। GitHub repository বা npm package-এর মতো বাইরের উৎস থেকে আসা যে plugin শুধু প্রকল্পের `.claude/settings.json`-এ চালু করা আছে, সেটি সদস্য নিজে ইনস্টল না করা পর্যন্ত লোড হবে না। ততক্ষণ Claude Code plugin-টিকে "ইনস্টল করা নেই" হিসেবে দেখায় এবং চালানোর জন্য `claude plugin install` command-টি দেখিয়ে দেয়।

৮.২ প্রতিষ্ঠানের নিয়ন্ত্রণ

প্রশাসকেরা managed settings দিয়ে নিয়ন্ত্রণ করতে পারেন ব্যবহারকারীরা কোন কোন marketplace যোগ করতে পারবেন:

Setting	কাজ
<code>strictKnownMarketplaces</code>	অনুমোদিত marketplace-এর তালিকা বেঁধে দেয় — এর বাইরে থেকে কিছু যোগ বা ইনস্টল করা যায় না
<code>pluginSuggestionMarketplaces</code>	কোন marketplace থেকে <code>/plugin</code> -এ ইনস্টলের পরামর্শ দেখানো হবে তা ঠিক করে
<code>extraKnownMarketplaces</code>	প্রতিষ্ঠান বা repository-র জন্য marketplace নিবন্ধন করে; প্রতিটি entry-তে <code>"autoUpdate": true</code> দিলে auto-update চালু হয়
<code>enabledPlugins</code>	কোন plugin চালু বা বন্ধ থাকবে তা scope অনুযায়ী নির্ধারণ করে

`strictKnownMarketplaces` চালু থাকলে **Not used recently** শিরোনাম ও **Last used** লাইনটি লুকিয়ে রাখা হয়।

Managed scope-এ ইনস্টল করা plugin ব্যবহারকারী বদলাতে পারেন না — `/plugin` -এ সেগুলো **managed** হিসেবে দেখানো হয়।

৮.৩ নিজের marketplace তৈরি ও বিতরণ

Marketplace আসলে একটি git repository, যার মধ্যে `.claude-plugin/marketplace.json` ফাইলে plugin-গুলোর তালিকা থাকে। প্রতিষ্ঠানের ভেতরের plugin গোপন রাখতে চাইলে marketplace-টি একটি **private repository**-তে রাখুন।

Plugin প্রস্তুত হলে বিতরণের আগে করণীয়:

- 1 **ডকুমেন্টেশন লিখুন** — একটি `README.md`, যাতে ইনস্টল ও ব্যবহারের নির্দেশনা থাকে
- 2 **Version কৌশল ঠিক করুন** — `version` ক্ষেত্রটি নিজে দেবেন, নাকি ডিফল্ট ব্যবস্থার উপর ছেড়ে দেবেন
- 3 **Marketplace-এ যুক্ত করুন** — নিজের marketplace তৈরি করুন বা বিদ্যমান কোনোটিতে জমা দিন
- 4 **অন্যদের দিয়ে পরীক্ষা করান** — বড় পরিসরে ছাড়ার আগে দলের সদস্যদের দিয়ে যাচাই করিয়ে নিন

৮.৪ Community marketplace-এ plugin জমা দেওয়া

জমা দেওয়ার জন্য অ্যাপের ভেতরের যেকোনো একটি form ব্যবহার করুন:

- **claude.ai** — claude.ai/admin-settings/directory/submissions/plugins/new
এর জন্য Team বা Enterprise প্রতিষ্ঠান এবং directory management-এর অনুমতি লাগে; প্রতিষ্ঠানের Owner-রা এই অনুমতি ডিফল্টভাবেই পান।
- **Console** — platform.claude.com/plugins/submit
যাঁরা কোনো Team বা Enterprise প্রতিষ্ঠানের অংশ নন, তাঁরা এটি ব্যবহার করতে পারেন।

জমা দেওয়ার আগে অবশ্যই স্থানীয়ভাবে `claude plugin validate ./your-plugin` চালিয়ে নিন — পর্যালোচনার সময় ঠিক এই পরীক্ষাটিই চালানো হয়, সঙ্গে স্বয়ংক্রিয় নিরাপত্তা যাচাই।

অনুমোদিত plugin community ক্যাটালগে একটি নির্দিষ্ট **commit SHA**-তে pin করা হয়, এবং আপনি নতুন commit push করলে CI নিজেই সেই pin হালনাগাদ করে দেয়। প্রকাশ্য ক্যাটালগটি প্রতি রাতে sync হয়, তাই অনুমোদনের পরে plugin দেখা যেতে কিছুটা সময় লাগতে পারে।

সমস্যা সমাধান

Plugin নিয়ে কাজ করতে গিয়ে যে সমস্যাগুলো সবচেয়ে বেশি হয়, সেগুলোর কারণ ও সমাধান।

৯.১ /plugin command চেনা যাচ্ছে না

"unknown command" দেখালে বা `/plugin` তালিকায় না এলে:

1 সংস্করণ দেখুন:

```
claude --version
```

2 Claude Code হালনাগাদ করুন:

```
# Homebrew
brew upgrade claude-code

# npm
npm install -g @anthropic-ai/claude-code@latest
```

Native installer দিয়ে ইনস্টল করে থাকলে Setup-এর install command-টি আবার চালান।

3 টার্মিনাল বন্ধ করে আবার খুলে `claude` চালান।

`/plugin` একটি interactive panel খোলে, যা টার্মিনাল CLI-র জন্য। Claude যদি বলে এই পরিবেশে `/plugin` নেই, তাহলে Desktop app-এর plugin browser ব্যবহার করুন, অথবা cloud session-এর জন্য `.claude/settings.json`-এ `enabledPlugins` ঘোষণা করুন (অধ্যায় ৫.৩)।

৯.২ ইনস্টল ব্যর্থ হচ্ছে

বার্তা	করণীয়
Marketplace "claude-plugins-official" not found	Marketplace-টি যোগ করে আবার চেষ্টা করুন: <code>/plugin marketplace add anthropics/claude-plugins-official</code>
Plugin marketplace-এ পাওয়া যাচ্ছে না	নামের বানান মিলিয়ে দেখুন। নতুন প্রকাশিত plugin হলে আগে <code>/plugin marketplace update <name></code> চালান।
marketplace not refreshed	ইনস্টলের আগে ক্যাটালগ হালনাগাদ করা যায়নি (যেমন ইন্টারনেট নেই) — cache করা ক্যাটালগ থেকেই ইনস্টল হয়েছে।

৯.৩ Skill দেখা যাচ্ছে না

Plugin ইনস্টল হয়েছে কিন্তু তার skill আসছে না — cache পরিষ্কার করে দেখুন:

```
rm -rf ~/.claude/plugins/cache
```

তারপর Claude Code আবার চালু করে plugin-টি পুনরায় ইনস্টল করুন।

আরেকটি সম্ভাবনা: ইনস্টলের সারসংক্ষেপে `Run /reload-plugins to activate.` লেখা ছিল, কিন্তু সেটি চালানো হয়নি।

৯.৪ Code intelligence (LSP) সংক্রান্ত সমস্যা

সমস্যা	সমাধান
Language server চালু হচ্ছে না <code>Executable not found in \$PATH</code>	Binary-টি ইনস্টল আছে কি না এবং <code>PATH</code> -এ আছে কি না দেখুন (তালিকা অধ্যায় ৪.১-এ)। বিস্তারিত <code>/plugin</code> -এর Errors ট্যাবে।
অতিরিক্ত memory খরচ	<code>rust-analyzer</code> ও <code>pyright</code> -এর মতো server বড় প্রকল্পে অনেক memory নেয়। সমস্যা হলে <code>/plugin disable <plugin-name></code> দিয়ে বন্ধ করে Claude-এর নিজস্ব search tool-এর উপর নির্ভর করুন।
Monorepo-তে ভুল diagnostics	Workspace ঠিকমতো কনফিগার না থাকলে ভেতরের package-এর import "unresolved" দেখাতে পারে। এতে Claude-এর সম্পাদনার ক্ষমতা কমে না।
কনফিগারেশন ভুল	ভুল কনফিগারেশনের entry নীরবে বাদ পড়ে যায়। কারণ জানতে <code>claude --debug</code> চালান।

৯.৫ Diagnostics নিজে পড়া

Claude Code যখন **Found 3 new diagnostic issues in 2 files**-এর মতো বার্তা দেখায়, তখন **Ctrl+O** চেপে আপনি নিজেই সেই diagnostics পড়তে পারেন।

Command রেফারেন্স

এই পাতাটি আলাদা করে প্রিন্ট করে হাতের কাছে রাখতে পারেন।

Session-এর ভেতরের command

Command	কাজ
<code>/plugin</code>	Plugin manager খোলা (Discover / Installed / Marketplaces / Errors)
<code>/plugin install <name>@<market></code>	Plugin ইনস্টল করা
<code>/plugin list</code>	ইনস্টল করা plugin-এর তালিকা
<code>/plugin list --enabled</code>	শুধু চালু plugin
<code>/plugin list --disabled</code>	শুধু বন্ধ plugin
<code>/plugin info <name></code>	Plugin-এর বিস্তারিত
<code>/plugin enable <name>@<market></code>	বন্ধ plugin আবার চালু করা
<code>/plugin disable <name>@<market></code>	না মুছে বন্ধ রাখা
<code>/plugin uninstall <name>@<market></code>	সম্পূর্ণ মুছে ফেলা
<code>/plugin marketplace add <source></code>	Marketplace যোগ করা
<code>/plugin marketplace list</code>	Marketplace-এর তালিকা
<code>/plugin marketplace update <name></code>	ক্যাটালগ হালনাগাদ
<code>/plugin marketplace remove <name></code>	Marketplace বাদ দেওয়া (এর plugin-ও মুছে যায়)
<code>/reload-plugins</code>	পরিবর্তন কার্যকর করা, restart ছাড়াই
<code>/reload-plugins --force</code>	Prompt cache বাতিল হলেও জোর করে reload
<code>/plugin-name:skill-name</code>	Plugin-এর skill ব্যবহার করা

সংক্ষেপ: `/plugin marketplace` → `/plugin market`, এবং `remove` → `rm`

Shell command

Command	কাজ
<code>claude plugin init <name></code>	নতুন plugin-এর কাঠামো তৈরি

Command	কাজ
<code>claude plugin init <name> --with ...</code>	নির্দিষ্ট উপাদান সহ কাঠামো তৈরি
<code>claude plugin install <plugin></code>	Interactive ধাপ ছাড়াই ইনস্টল
<code>claude plugin install ... --scope project</code>	নির্দিষ্ট scope-এ ইনস্টল
<code>claude plugin list</code>	তালিকা (<code>--json</code> , <code>--available</code> সহ)
<code>claude plugin details <plugin></code>	কোন কোন উপাদান যোগ করে তার তালিকা
<code>claude plugin enable / disable</code>	চালু বা বন্ধ করা
<code>claude plugin update <plugin></code>	হালনাগাদ করা
<code>claude plugin uninstall <plugin></code>	মুছে ফেলা (<code>--keep-data</code> , <code>--prune</code> সহ)
<code>claude plugin validate ./my-plugin</code>	Plugin যাচাই করা (<code>--strict</code> সহ)
<code>claude plugin marketplace add <source></code>	Shell থেকে marketplace যোগ
<code>claude --plugin-dir ./my-plugin</code>	স্থানীয় plugin লোড করে চালানো
<code>claude --plugin-url <url.zip></code>	URL-এর .zip থেকে লোড করা
<code>claude --debug</code>	বিস্তারিত ত্রুটির তথ্য দেখা
<code>claude --version</code>	সংস্করণ দেখা

পরিভাষা

এই বইয়ে ব্যবহৃত ইংরেজি পরিভাষাগুলোর বাংলা ব্যাখ্যা।

ইংরেজি	বাংলা ব্যাখ্যা
Plugin	স্বয়ংসম্পূর্ণ প্যাকেজ, যা Claude Code-এ নতুন সক্ষমতা যোগ করে
Marketplace	Plugin-এর ক্যাটালগ, যেখান থেকে খুঁজে ইনস্টল করা যায়
Manifest	<code>plugin.json</code> ফাইল, যেখানে plugin-এর পরিচয় ও কনফিগারেশন থাকে
Skill	নির্দিষ্ট কাজের জন্য লেখা নির্দেশনা; Claude প্রসঙ্গ বুঝে নিজেই ব্যবহার করে
Agent / Subagent	নিজস্ব prompt, model ও tool-সীমাসহ বিশেষায়িত সহকারী
Hook	নির্দিষ্ট ঘটনার সঙ্গে বাঁধা script, যা স্বয়ংক্রিয়ভাবে চলে
MCP server	Model Context Protocol server — বাইরের service-এর tool যোগ করে
LSP server	Language Server Protocol server — কোড বোঝা ও type error ধরার জন্য
Namespace	নামের সংঘর্ষ এড়াতে ব্যবহৃত উপসর্গ, যেমন <code>/plugin-name:skill-name</code>
Scope	Plugin কোথায় কার্যকর হবে তার পরিধি — user, project, local বা managed
Context window	Claude একসঙ্গে যতটুকু তথ্য মনে রাখতে পারে; প্রতিটি plugin এর কিছু অংশ দখল করে
Context cost	একটি plugin প্রতি turn-এ যত token যোগ করে
Diagnostics	Language server-এর জানানো error ও warning
Output style	Claude কীভাবে উত্তর সাজাবে তার ধরন
Monitor	পটভূমিতে চলা প্রক্রিয়া, যা log বা অবস্থা দেখে Claude-কে জানায়
Synced plugin	claude.ai অ্যাকাউন্টে চালু করা plugin, যা cloud session-এ লোড হয়
Managed settings	প্রতিষ্ঠানের প্রশাসকের নির্ধারিত setting, যা ব্যবহারকারী বদলাতে পারেন না

তথ্যসূত্র — এই বইয়ের সব তথ্য Claude Code-এর সরকারি ডকুমেন্টেশন থেকে নেওয়া: code.cloude.com/docs/en/plugins/, [/discover-plugins/](https://code.cloude.com/docs/en/discover-plugins/), [/plugins-reference/](https://code.cloude.com/docs/en/plugins-reference/), [/plugin-marketplaces/](https://code.cloude.com/docs/en/plugin-marketplaces/), [/desktop](https://code.cloude.com/docs/en/desktop/) এবং [/settings-reference](https://code.cloude.com/docs/en/settings-reference/)।

Claude Code দ্রুত বদলায় — কোনো command বা আচরণ এখানকার বর্ণনার সঙ্গে না মিললে `claude --version` দিয়ে সংস্করণ মিলিয়ে নিন এবং উপরের ডকুমেন্টেশন দেখুন। সংস্করণ ১.০ · সেপ্টেম্বর ২০২৬।